Data Structures (CSN12101)
ASSIGNMENT-8

Name : Aayansh Niranjan Date :25/03/2026
Reg_No.: 20253004

Q1. You must represent two stacks in an array, memory[MEMORY SIZEJ, Write C functions that add (PUSH) and delete (POP) an item from stack i, 0<j<n. Your functions should be able to add elements to the stacks as long as the total number of elements in both stacks is less than MEMORY SIZE-1.

A1. CODE:

```
#include <stdio.h>
#define SIZE 100

int arr[SIZE];
int top1=-1,top2=SIZE;

void push1(int x){
   if(top1+1==top2){
      printf("Stack Overflow\n");
      return;
   }
   arr[++top1]=x;
}

void push2(int x){
   if(top1+1==top2){
      printf("Stack Overflow\n");
      return;
   }
   arr[--top2]=x;
}

int pop1(){
   if(top1==-1){
      printf("Stack 1 Underflow\n");
      return -1;
   }
   return arr[top1--];
}

int pop2(){
   if(top2==SIZE){
      printf("Stack 2 Underflow\n");
      return -1;
   }
   return arr[top2++];
}
```

```
void display(){
    printf("Stack1: ");
    for(int i=0;i<=top1;i++) printf("%d ",arr[i]);
    printf("\nStack2: ");
    for(int i=SIZE-1;i>=top2;i--) printf("%d ",arr[i]);
    printf("\n");
}

int main(){
    push1(10);
    push1(20);
    push2(100);
    push2(200);
    display();
    printf("Pop Stack1: %d\n",pop1());
    printf("Pop Stack2: %d\n",pop2());
    display();
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_8$ ./p1
Stack1: 10 20
Stack2: 100 200
Pop Stack1: 20
Pop Stack2: 200
Stack1: 10
Stack2: 100
```

Q2. Given a singly linked list head with n nodes, write a function Rotate_K(head, k) that rotates every group of k nodes to the right in the linked list. The function should return the modified linked list. If the number of nodes is not a multiple of k, leave the remaining nodes as is. Rotation means shifting each node in the group one position to the right, with the last node moving to the front of the group.

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
    int data;
    struct Node* next;
};

struct Node* createNode(int x){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=x;
    newNode->next=NULL;
    return newNode;
}

void append(struct Node** head,int x){
    struct Node* temp=createNode(x);
    if(*head==NULL){
        *head=temp;
        return;
    }
    struct Node* curr=*head;
    while(curr->next) curr=curr->next;
    curr->next=temp;
}

void printList(struct Node* head){
    while(head){
        printf("%d ",head->data);
        head=head->next;
    }
    printf("\n");
}

struct Node* rotateK(struct Node* head,int k){
    if(!head||k<=1) return head;
    struct Node dummy;
    dummy.next=head;
    struct Node* prevGroupEnd=&dummy;
```

```
    while(1){
        struct Node* kth=prevGroupEnd;
        for(int i=0;i<k&&kth;i++) kth=kth->next;
        if(!kth) break;
        struct Node* groupStart=prevGroupEnd->next;
        struct Node* groupEnd=kth;
        struct Node* nextGroup=groupEnd->next;
        struct Node* secondLast=groupStart;
        while(secondLast->next!=groupEnd) secondLast=secondLast->next;
        secondLast->next=nextGroup;
        groupEnd->next=groupStart;
        prevGroupEnd->next=groupEnd;
        prevGroupEnd=secondLast;
    }
    return dummy.next;
}

int main(){
    struct Node* head=NULL;
    int arr[]={1,2,3,4,5,6,7,8,9},n=9;
    for(int i=0;i<n;i++) append(&head,arr[i]);
    printf("Original List:\n");
    printList(head);
    head=rotateK(head,3);
    printf("After Rotation (k=3):\n");
    printList(head);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_8$ ./p2
Original List:
1 2 3 4 5 6 7 8 9
After Rotation (k=3):
3 1 2 6 4 5 9 7 8
```